PolyPaint: Architecture & UI Critique

Current state, trade-offs, and concrete alternatives

Prepared from a full-tree review

June 2026

Executive summary

PolyPaint is a working end-to-end system with unusually strong *verification culture*: cross-language drift tests, payload-contract tests, a Docker runtime gate, fingerprint-stable wire formats, and a deploy script that converges instead of hoping. The architecture's defining choice is **radical buildlessness** — one HTML file, one bash deployer, zips of plain Python around native ARM binaries — and most observed costs are the predictable tax of that choice rather than accidents.

Three findings dominate:

1. **The four-representation program pipeline (text → chips → tokens → C VM, mirrored again in the JS catalog) is the system's largest standing risk.** Every recent defect class — alias drift, canonicalization gaps, layout-order flips — is a synchronization failure between copies of the same fact. The fix is not a rewrite; it is generating the mirrors from one machine-readable catalog.
2. **The UI is information-dense but workflow-poor.** It shows every knob of one stage at a time, while the actual creative loop (edit program → compute → preview → render → compare) crosses four tabs and lives partly in the user's memory. "Constraining" is the right word: the layout mirrors the wire format, not the loop.
3. **The biggest UI unlocks are cheap.** A persistent jobs rail, a click-to-inspect chip editor, side-by-side text ↔ chips, and an XY scrub pad driving the *already-deployed* lores-preview Lambda would transform feel without touching the backend.

1 System inventory

Numbers measured on the current tree (post CR1–CR7 fixes, deployed).

Component	Size
Frontend (index.html, vanilla JS, no build)	20,033 lines
JS functions in frontend	774
Backend Lambda handlers (Python 3.12, arm64)	40 files
Python modules under lambda/	89 files
Native C sources (solver, raster, codecs)	39 files, 167k lines
Largest C source (sweep_cli.c: solver + 2 program VMs)	9,956 lines
Deployed Lambda functions (single spec list)	41
API Gateway routes (POST)	49
Step Functions state machines	3
deploy.sh (idempotent create/update, gates inline)	2,144 lines
Test files / passing tests	113 / 1,279 (+33 subtests)
Program DSLs (param + coeff), text and chip front ends	2

2 Architecture critique

Each decision below gets the same treatment: what it is, what it buys, what it costs, real alternatives, and a verdict. "Replace" appears rarely — most of this architecture is earning its keep.

2.1 D1 — One 20k-line HTML file, vanilla JS, no build step

Pros. Deploy is `aws s3 cp` with a hash check; zero dependency churn or supply-chain surface; everything greppable in one file; the test harness can extract and execute real source; no framework abandonment risk; instant cold load (one fetch).

Cons. No module system, so isolation is by naming convention (`_coeffProgram*`) and load-order/TDZ guards that look like noise until removed; 774 functions share one namespace; no static analysis — the serializer self-recursion shipped because nothing type-checks or lint-checks the file; editing is index-arithmetic at 20k lines; review diffs concentrate in one path.

Alternatives.

1. *ES modules, still no bundler.* Split into `js/*.mjs` loaded via `<script type="module">`; S3 serves them fine. Real imports kill the TDZ-guard idiom and give the harness import-level seams. Cost: a deliberate state-object pass (the implicit global state must become explicit), cache-busting on uploads. Medium effort, permanent payoff.
2. *Keep one file, add checking.* JSDoc type annotations + `tsc --checkJs` + ESLint (`no-self-compare`, recursion and shadow rules) wired into `predeploy_check.sh`. Catches the bug classes actually observed. Days, not weeks.
3. *A framework (React/Svelte/Vue).* Componentized chips and panels, but imports a build pipeline, a dependency treadmill, and a full state-management rewrite into a codebase whose value is determinism. Weakest option here.

Verdict: Keep the buildless philosophy; do alternative 2 now and treat alternative 1 as the next structural step. Skip 3.

2.2 D2 — bash deploy.sh as the infrastructure layer

Pros. Readable end to end; no opaque CloudFormation state or drift mystery; converge semantics (create falls through to update and vice versa); `show-build` verifies deployed-vs-local instead of

assuming; after the CR5 unification there is exactly one Lambda spec list, structure pinned by tests.

Cons. 2,144 lines of bash is a bespoke platform: no plan/diff before apply, no changesets or rollback, IAM hand-rolled, no second environment without hand-cloning names/buckets; failure mid-deploy leaves a partially-updated stack (mitigated, not solved, by idempotence).

Alternatives.

1. *Terraform/CDK/SAM wholesale.* Plan-before-apply and environments for free, but you trade a transparent script for state files and abstractions, re-encode 41 functions + 3 ASL templates + IAM, and lose the inline gates. High cost, moderate payoff at this team size.
2. *Manifest-driven bash.* Keep the engine, move the data out: the spec list, routes, and env maps become one JSON (the repo already has `api_manifest.json` as a half-step); deploy.sh iterates it, tests validate it, and the same JSON could later feed any IaC tool. Low cost, kills remaining list-drift classes.
3. *Hybrid:* Terraform for the slow-moving substrate (bucket, table, roles, API), script for the fast-moving Lambda/ASL layer.

Verdict: Keep, with alternative 2. Revisit IaC only when a second environment or a second operator becomes real.

2.3 D3 — 41 single-purpose Lambdas, shared code copied into each zip

Pros. Per-function memory/CPU tuning (10,240 MB solvers next to 512 MB metadata routes — this is real money); blast-radius isolation; reserved concurrency protects storage/dispatch from render storms; per-function logs.

Cons. 41 zips × (handler, deps, env, layers, tmp, IAM, route, permission) is wiring that must be tested to stay true — the packaging tests exist *because* the surface is wide; `shared.py` and friends are *copied* into every zip, so two Lambdas can run different versions of the same module mid-deploy; 41 cold starts; per-zip 250 MB limits force the layer dance for libvips/LAPACK.

Alternatives.

1. *Shared-code layer.* One `polypaint-common` layer for the pure-Python helpers; zips shrink to handler + binaries; version skew window collapses to the layer publish. Low effort; the layer mechanism is already in daily use.
2. *One container image, many handlers.* A single ECR image with every handler and binary; each function differs only by `CMD`. Kills zip drift entirely, slower publishes, 10 GB budget. Medium effort.
3. *Consolidation into role-based Lambdas* (sync API, async worker, orchestration): fewer moving parts but loses per-function tuning — the 10 GB/512 MB split is worth keeping.
4. *Fargate for the long renders*: removes the 15-minute ceiling if it ever binds; not currently binding.

Verdict: Keep the topology; adopt alternative 1 (shared layer) — it is the cheapest fix for the realest risk (version skew).

2.4 D4 — Python handlers around native C binaries via JSON-over-stdin

Pros. The solver runs at native speed with LAPACK where it counts; process isolation means a solver crash is a clean Python exception, not a corrupted runtime; the JSON protocol is observable

(any spec replays from a shell); Python stays in charge of S3/DynamoDB where it belongs.

Cons. Cross-language duplication is structural: opcodes, limits, selector enums, and registry ids exist in Python, C, and JS, held together by drift tests (a tax, paid correctly, but a tax); process spawn + serialization per invocation; builds require a Docker round-trip inside deploy; binary staleness is a real failure mode (hence the mtime gate).

Alternatives.

1. *Generate the mirrors.* One `vm_spec.json` (opcodes, limits, fn indices, aliases) from which a small generator emits the C header, the Python constants, and the JS tables — `gen_catalog` already proves the pattern in this repo. Drift tests then verify generated output instead of hand-copies. This is the highest-leverage architecture change available.
2. *In-process via cffi/ctypes.* Load the VM as a shared library: no spawn, no JSON hop. Loses crash isolation and complicates the layer story; only worth it if per-call overhead ever shows in profiles.
3. *Rust + PyO3.* One toolchain, memory safety, good cross-compile. A future, not a refactor: 10k lines of validated C with 40 WASM-parity tests is an asset, not a liability.

Verdict: Keep the split; do alternative 1. Treat 2 and 3 as profile-driven, not aesthetic, decisions.

2.5 D5 — Two stack-machine DSLs with text and chip front ends

Pros. Fingerprintable, deterministic, bounded programs (no `eval`, no user JS in Lambdas); text mode for power, chips for discoverability, one compiler under both; macros enable composition; the typed stack + scalar-expression bytecode split cleanly separates stack values from per-row arguments (now documented as design).

Cons. Four representations of one program (text, chip rows, binary tokens, JS catalog) and *two* sibling-but-different VMs (param vs coeff) multiply every concept; the recent review cycle (CR2, CR6, CR7) was dominated by mirror-sync bugs — aliases, canonical names, src/tgt order; chips serialize wire rows rather than user intent, which leaks all the way into the UI (Section 4).

Alternatives.

1. *Single catalog source of truth* (same move as D4-1): chip defs, aliases, param shapes, andy arity, selector vocab in one JSON; JS catalog and Python tables generated. Eliminates the CR6/CR7 bug class rather than patching instances of it.
2. *Unify the two VMs*: one token VM with two register files (param scalars, coeff vectors). Conceptually right, but fingerprints are wire format — this is a migration with versioned specs, not a refactor. Queue behind a real feature need.
3. *Adopt an existing language* (Lua, a JS subset): more expressive, but forfeits determinism, fingerprint stability, and the bounded execution that makes Lambda-side eval safe. Wrong trade here.
4. *Make text the only source* and render chips as a read-only visualization: halves the editing surface; viable only after the UI in Section 5.2 exists.

Verdict: Keep the DSL design; invest in alternative 1 immediately, hold 2 until a feature forces a fingerprint version bump anyway.

2.6 D6 — Fingerprint-addressed artifact cache, frozen wire formats

Pros. Identical work is never recomputed; cache keys are content-derived so correctness does not depend on invalidation logic; byte-stability discipline made several refactors provably safe (verified against HEAD during the cleanup pass).

Cons. The freeze constrains internals forever (the `_execution_spec` printer cannot change cosmetically); a historical quirk — the src/tgt order flip between row layouts — is now permanent; nothing in the key encodes *format generation*, so any future change is all-or-nothing.

Alternatives. Add a `spec_version` field folded into the fingerprint: old artifacts stay valid under v1 while v2 can fix layout quirks; dual-read during transition. Cheap insurance, no behavior change today.

Verdict: Keep; add the version field at the next natural migration.

2.7 D7 — Step Functions + dispatch Lambda + DynamoDB status + UI polling

Pros. Standard workflows give visible execution history, retries, and state-size discipline; DynamoDB rows with TTL are a clean progress ledger; the dispatch Lambda centralizes async fan-out targets in one env map (single spec list, post-CR5).

Cons. ASL templating via `sed` substitution in deploy is stringly-typed (mitigated by shared renderers + definition tests); payload-contract drift across planner/ASL/worker is the checklist's heaviest section for a reason; the UI polls — status latency and request noise — and shows text, not the DAG that Step Functions already knows.

Alternatives. (1) WebSocket API or IoT-Core topic pushing status deltas: kills polling, enables the jobs rail in Section 5.3; medium effort. (2) EventBridge-driven choreography instead of orchestration: more decoupled, much harder to see; not recommended — visibility is this system's strength. (3) Surface the existing SFN execution history directly in the UI (read-only API call) before building anything new.

Verdict: Keep; do alternative 3 cheaply, then 1 when the jobs rail lands.

2.8 D8 — The verification stack

Pros. This is the standout: enum/limit drift tests across three languages; packaging tests that parse deploy.sh; payload-shape contract tests; a Docker gate that runs the real ARM binaries; a frontend harness that now *executes* extracted source (the recursion bug forced that upgrade); fingerprint-stability checks against HEAD during refactors.

Cons. The JS harness is homegrown extract-and-pin — exact-text pins mean edits to pinned lines must update the harness in the same change (a feature for wire format, friction elsewhere); no browser-level E2E, so DOM wiring (onclick strings, element ids) is tested only by convention; property-based testing is absent where it would shine (random programs: Python fold vs C eval parity).

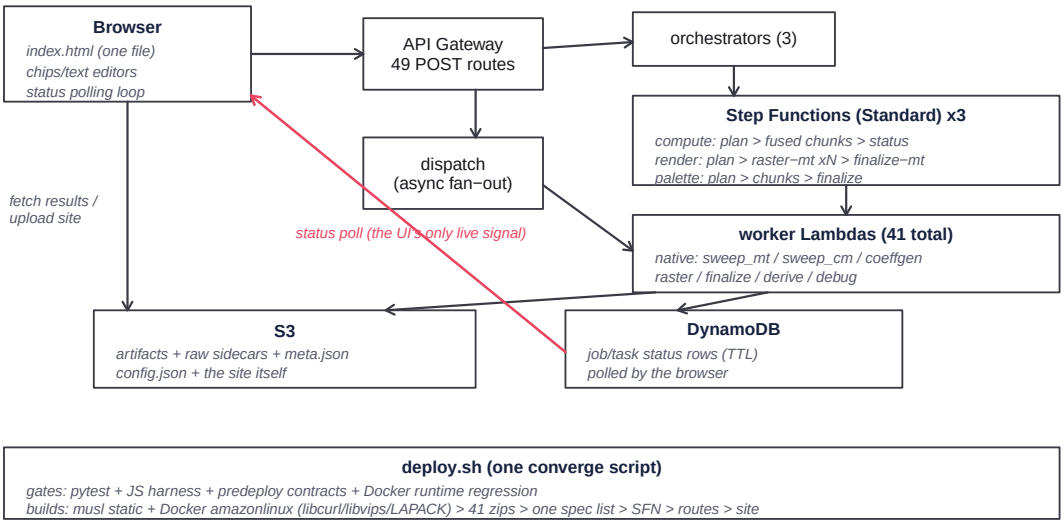
Alternatives. A 10-case Playwright smoke (load, add chip, compute preview, render, fetch result) covering the DOM seam; Hypothesis-based program generators for VM parity; after an ES-module split, real unit imports replace source extraction.

Verdict: Keep and extend: Playwright smoke + property tests are the two missing legs.

2.9 Decision matrix

Decision	Call	Risk if ignored
D1 buildless single-file UI	Keep + add checking	more unexecuted-path bugs
D2 bash deployer	Keep + manifest-drive	second-env pain
D3 41 small Lambdas	Keep + shared layer	mid-deploy version skew
D4 C core via subprocess	Keep + generate mirrors	three-way drift tax grows
D5 two DSLs, four mirrors	Keep + one catalog	next CR6-class bug
D6 fingerprint cache	Keep + spec version	frozen quirks accumulate
D7 SFN + polling	Keep + push status	polling UX ceiling
D8 verification stack	Keep + E2E/property	DOM-seam regressions

3 The workflow as built



Pain points in the loop, as experienced: the browser's only live signal is a poll against status rows (accent arrow above); job topology is invisible in the UI even though Step Functions has it; and the creative iteration loop (edit → preview → commit → compare) crosses four tabs with no persistent job context.

4 The UI as built

One dark-theme page, six tabs. The Compute tab is a vertical stack of rows; programs are edited as inline "chips" with small parameter boxes; the Coeff Program additionally has a Chips/Text toggle. Status is text lines under each box.

4.1 What works

Density without navigation depth — every control is one click away; chips render as readable formulas, not opaque IDs; the text editor with a real compiler and diagnostics is a power feature most tools never get; dark theme and accent discipline are consistent; nothing lies — status text reflects the actual contract.

4.2 Why it feels constraining

1. **The layout mirrors the wire format, not the workflow.** Each row is a payload field. The loop you actually run (edit → preview → render → compare) is spread across Compute, Render, and Results tabs that cannot be seen together.
2. **Stack programs in a flat strip.** RPN chips hide the one thing that matters — stack depth and what each chip consumes/produces. You simulate the machine in your head; the compiler knows but the UI doesn't show it.
3. **Editing in 12-character boxes.** Scalar expressions like `log(abs(p1+p2)+1)*1j` are edited in tiny inline inputs; the `*Wide` CSS classes are band-aids on a structural problem (no inspector surface).
4. **Binary Chips/Text toggle.** The two views never coexist, so text users lose chip affordances and chip users never learn the text idiom. Param Program doesn't get a text mode at all — an asymmetry with no user-facing reason.
5. **Catalog browsing by native `<select>`.** 28 coeff transforms in a dropdown; pickers are popup lists without search-as-you- type, previews, or examples.
6. **Jobs are log lines.** Long pipelines report through appended text; there is no persistent surface answering "what is running, how far along, what did it produce" — despite the backend knowing exactly that.

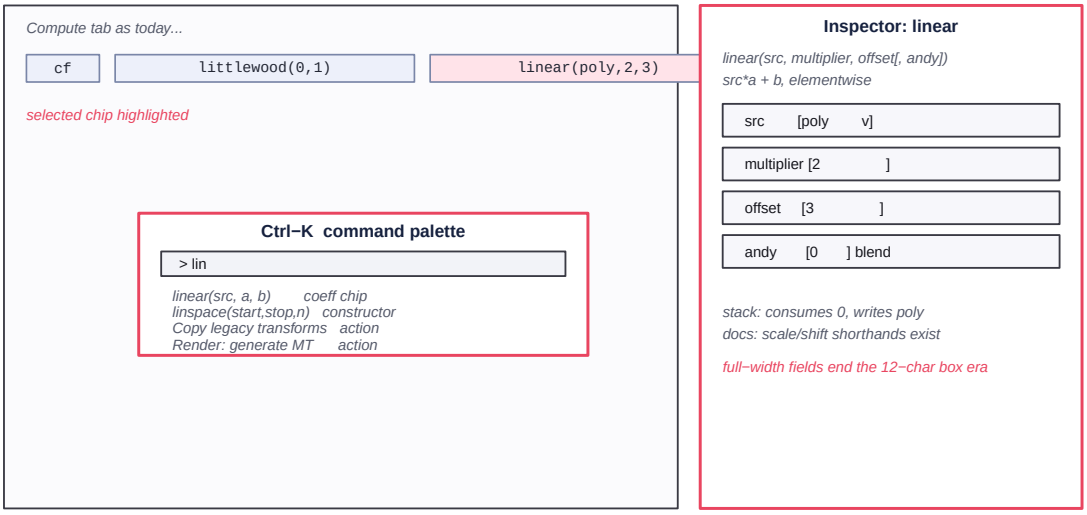
7. **No direct manipulation of parameters.** `p1/p2/t1/t2` exploration is type-a-number-and-recompute, although a lores preview Lambda exists that could make dragging live.
8. **Fixed geometry.** No panel resize, no collapse memory beyond preview tabs, no layout choice for wide monitors.

5 UI alternatives

Ordered roughly by effort. A–D keep the existing DOM and grow it; E–F are structural.

5.1 A — Command palette + chip inspector (days)

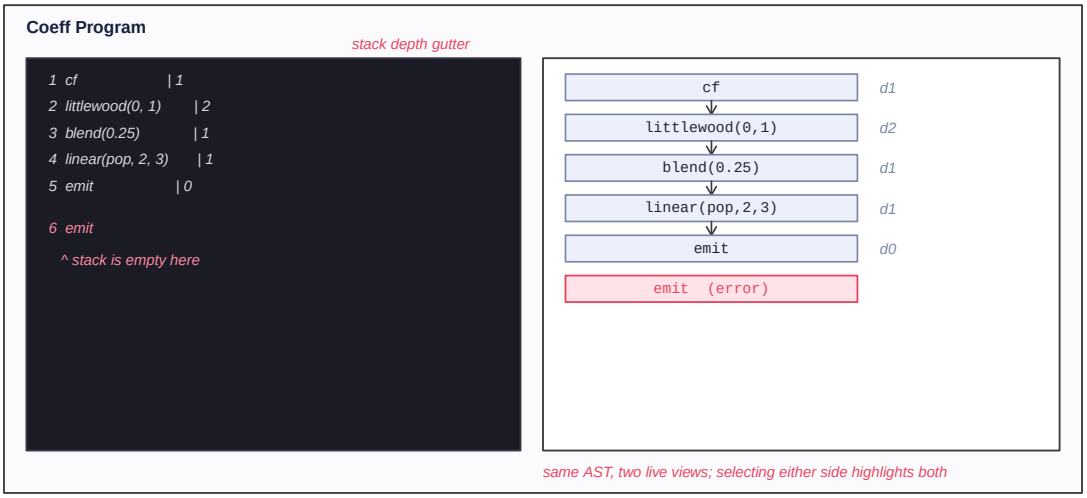
Two additive features. `Ctrl-K` opens a fuzzy palette over *every* action and catalog entry (add chip, switch tab, run compute, open saved program). Clicking any chip opens a right-side inspector with full-width labeled fields, the chip's doc line, and stack effect — the tiny inline boxes stay for glanceability but stop being the only editor.



Cost/benefit. No layout change, no backend change; the inspector reuses the existing catalog metadata verbatim. Highest value-per-line-of-code available.

5.2 B — Unified program editor: text and chips side by side (1–2 weeks)

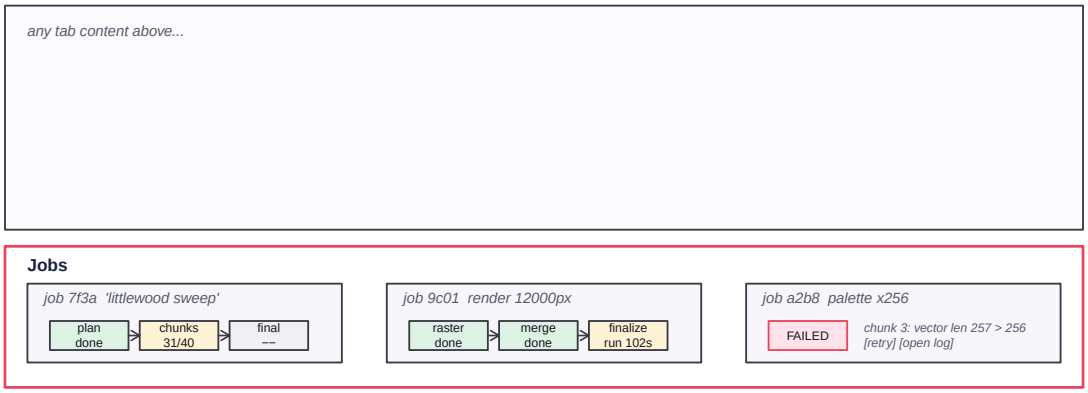
Kill the mode toggle. One pane shows text, the other chips; both are views of the same parsed program, edits in either round-trip live through the existing compiler. A gutter shows per-statement stack depth (the compiler already computes `stack_max`); errors annotate the offending line *and* chip. Param Program gets text mode for free since it shares the machinery.



Cost/benefit. Pure frontend; the compiler endpoints exist (`/compile-coeff-program-source` round-trips today). Makes the stack visible, ends the toggle, and converts the text editor from expert escape hatch into the primary surface with training wheels.

5.3 C — Jobs rail: the pipeline as a first-class surface (1 week)

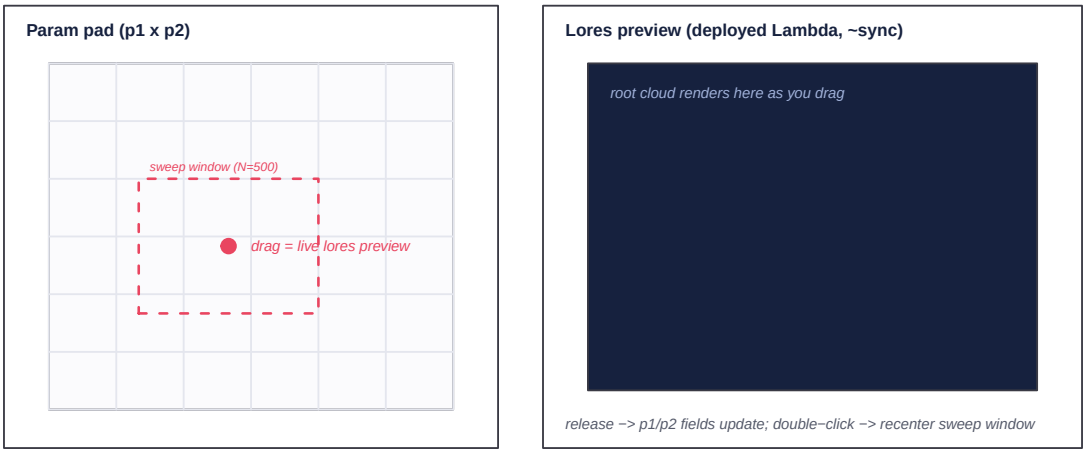
A persistent bottom strip across all tabs: one card per job showing the compute→render→finalize stages with live per-stage state from the existing status rows (later, pushed over WebSocket). Click a stage for its log slice and outputs; click the artifact to jump to Results with it selected. The log lines remain, demoted to a detail view.



Cost/benefit. Frontend + one read endpoint reshuffle; the data already exists in DynamoDB and SFN history. This single feature addresses the loudest "constraining" complaint — losing sight of work in flight.

5.4 D — Direct manipulation: XY pad + draggable markers (1 week)

Bind a 2-D pad to `(p1, p2)` with the grid extent drawn on it; drag to scrub — each move debounces into the deployed lores-preview Lambda and paints the small canvas; release to commit values back into the form. Scrubbing *is* exploration; the current sweep grid becomes a window you position by feel.



Cost/benefit. The expensive part (a fast preview path) is already deployed; this is wiring and debouncing. Highest delight-per-effort of the six.

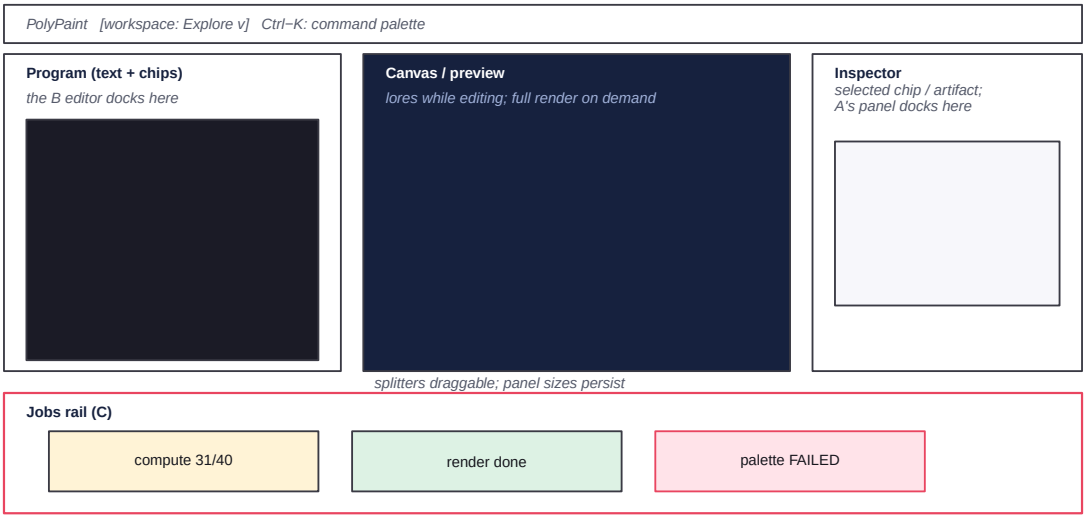
5.5 E — Gallery-first Results with a compare tray (1–2 weeks)

Replace row-list results with a thumbnail grid (the previews exist), faceted by function, palette, fingerprint family, and date; hover for metadata, click for detail, and a pinned "compare tray" that holds 2–4 artifacts side by side at matched zoom. Derivation actions (autolevels, repalette, bilevel, DeepZoom) move onto the detail view as buttons they already are.

Cost/benefit. Pure frontend over the existing `/list`-lean-then-`/detail` flow; turns Results from a filing cabinet into the place where judgment happens.

5.6 F — Studio shell: dockable three-pane layout (2–4 weeks)

The structural option: program editor left (the B editor), canvas/preview center, inspector right (the A inspector), jobs rail bottom (C). Tabs shrink to workspace presets rather than walls between stages. This is the layout the workflow has been asking for; the four prior pieces become its panels, which is the argument for building them first.



Cost/benefit. The only alternative requiring real layout surgery in the 20k-line file (or, more honestly, the moment the ES-module split from D1 pays for itself). Do it after A–D exist as panels.

Considered and not recommended (now). A node-graph editor (Blender-style) for the program DSLs: the stack machine is linear with occasional branches, and the B editor's depth gutter conveys the same information at a tenth of the cost. Revisit only if programs grow real DAG structure. Likewise a full framework rewrite (D1-3): nothing in A–F needs it.

6 Roadmap

Horizon	Item	Payoff
Now (days)	A: command palette + chip inspector (low risk)	ends tiny-box editing
Now (days)	tsc –checkJs + ESLint in predeploy (low)	catches the observed bug classes
Now (days)	shared-code Lambda layer (low)	kills mid-deploy version skew
Next (1–2 wks)	C: jobs rail — poll first, push later (low)	work in flight becomes visible
Next (1–2 wks)	D: XY scrub pad on lores preview (low)	exploration becomes tactile
Next (1–2 wks)	one catalog generates JS/Py/C tables (medium)	retires the CR6/CR7 drift class
Then (1–2 wks)	B: unified text + chips editor (medium)	stack visible; toggle dies
Then (1–2 wks)	E: gallery results + compare tray (low)	judgment surface for outputs
Then (2–4 wks)	F: studio shell on ES modules (high)	layout matches the loop

7 Closing

The architecture is coherent: buildless, observable, verified, and frozen where freezing pays (wire formats) — the critique is not "modernize" but "stop hand-copying facts" (one catalog, one shared layer, generated mirrors). The UI's bones are good and its density is a feature; what it lacks is the workflow layer — visible jobs, a real editing surface, and direct manipulation. Alternatives A, C, and D are small, independent, and each removes a distinct constraint; B and E complete the loop; F is the eventual shape once the panels exist. None of it requires touching the solver, the DSLs' semantics, or the deploy substrate — which is the strongest endorsement of the choices made that this review can offer.